Saga do atendimento NumberOne

Material para apresentar à equipe | 02/11/2026 | Matheus, Julio, Anderson e Marcelo

Proposta de complemento ao documento do Julio: manter sua jornada e fechar a coordenação, recuperação e compensações. Os estados, eventos adicionais e endpoints de compensação abaixo precisam de validação do grupo; ainda não são contratos implementados.

1. O fluxo que vamos preservar

Abrir OS → diagnosticar → aprovar orçamento → executar com consumo de peças → concluir reparo → pagar → entregar.

- OS controla clientes, veículos, diagnóstico, execução, entrega e coordenação da Saga.
- Billing calcula/aprova orçamento e integra pagamento Mercado Pago.
- Inventory controla saldo e movimentações; consumo por REST, sem reserva antecipada.
- Funcionário cria orçamento e pagamento pela API do Billing. Não acrescentar cobrança automática após conclusão.
- Billing publica decisões financeiras pelo SQS. OS consulta quitação no Billing antes da entrega.

A Saga não é uma transação de banco aberta durante todo o atendimento. A OS persiste a etapa global, recebe fatos e solicita as operações necessárias; cada serviço confirma somente sua própria transação. Ações humanas podem separar etapas por horas ou dias.

No exemplo, a OS-123 troca pastilhas: mão de obra R$ 150 + um kit R$ 200 = R$ 350. Inventory começa com cinco kits. Os IDs abreviados são ilustrativos; contratos usam UUID.

2. Onde cada serviço guarda o andamento

Serviço / banco	Dados que controla	Exemplo de estado local
OS / PostgreSQL próprio	Ordem, itens, Saga, operações de integração, inbox e histórico	Saga aguardando aprovação; início do item pendente; cancelamento em andamento
Billing / PostgreSQL próprio	Orçamento, pagamento, operações externas, inbox/outbox	Orçamento aprovado; pagamento pendente; estorno em processamento
Inventory / PostgreSQL próprio	Saldo, movimentos, operações de consumo/reversão	Consumo aplicado; operação encerrada/revertida; saldo insuficiente
OS / DynamoDB	Projeção consultável das revisões concluídas	Revisão do veículo publicada após conclusão

Estado global e decisões do orquestrador ficam na OS. Billing e Inventory implementam operações idempotentes, consultas de resultado e compensações dos seus efeitos. Ninguém acessa o banco de outro serviço.

O histórico DynamoDB mantém o uso NoSQL previsto anteriormente: snapshot/outbox PostgreSQL → worker → documento por veículo. Atraso nessa projeção não bloqueia pagamento/entrega nem dispara compensação do reparo.

3. Caminho principal, do início à entrega

Etapa	O que acontece	Comunicação / gravação
1	Equipe cadastra cliente/veículo e abre OS; OS cria acompanhamento da Saga	HTTP para OS; PostgreSQL OS
2	Equipe registra diagnóstico, serviços e materiais previstos	Operações locais da OS; sem mensagem de diagnóstico para Inventory
3	Funcionário cria orçamento no Billing; Billing consulta contexto técnico e preços, congela composição e calcula R$ 350	REST Billing → OS/Inventory; orçamento no PostgreSQL Billing
4	Billing envia orçamento; cliente aprova integralmente	Billing grava decisão + outbox; SQS OrcamentoEnviado/OrcamentoAprovado → OS
5	Equipe inicia OS aprovada e solicita início do item; OS grava operação CON-1 pendente antes de chamar Inventory	PostgreSQL OS; depois REST de consumo, fora da transação SQL
6	Inventory valida e baixa o conjunto de materiais atomicamente: cinco kits → quatro; salva movimento e resultado CON-1	Uma transação PostgreSQL Inventory; resposta REST
7	OS confirma início do item após resultado conhecido e revalidação de versão/cancelamento	PostgreSQL OS; equipe recebe autorização para executar
8	Equipe realiza e conclui o serviço; OS conclui somente com itens terminais e sem operações pendentes	HTTP/local OS; snapshot para histórico NoSQL
9	Funcionário escolhe PIX/cartão e cria pagamento no Billing; Billing consulta OS COMPLETED e chama Mercado Pago	REST; operação persistida no Billing antes do provedor
10	Billing verifica confirmação no provedor e registra PAID + outbox	PostgreSQL Billing; SQS PagamentoConfirmado → OS
11	Ao entregar, OS consulta settlement no Billing, mesmo que a projeção local já indique PAID	REST OS → Billing; falha/indefinição bloqueia entrega
12	Quitação confirmada e guardas locais satisfeitos: OS registra DELIVERED e conclui a Saga	PostgreSQL OS; histórico preservado

OrcamentoEnviado é uma adição proposta para resolver WAITING_APPROVAL. Os outros três fatos centrais já estão no PDF: OrcamentoAprovado, OrcamentoRejeitado e PagamentoConfirmado. Um item sem insumos não exige chamada vazia: OS pode iniciá-lo localmente, após as guardas comerciais.



As chamadas REST do diagrama retornam respostas mesmo onde não há seta de retorno desenhada. Nenhuma chamada HTTP permanece aberta aguardando o cliente aprovar, pagar ou executar o reparo.

4. Quais falhas exigem compensação?

Compensar é executar outra operação para neutralizar um efeito já confirmado. **Recuperar** é descobrir o resultado ou continuar uma etapa que falhou tecnicamente. Não são a mesma ação.

Situação	Tratamento	Classificação
Cliente rejeita orçamento	OS fica rejeitada/aguardando revisão; nada foi consumido/cobrado	Decisão comercial, sem estoque a devolver
Falta uma peça no consumo agrupado	Inventory não baixa nenhum item; OS marca espera por peças	Recusa sem efeito parcial; nova tentativa explícita após reposição
Inventory consumiu, mas resposta se perdeu	Manter início pendente; consultar/repetir CON-1 e confirmar resultado	Recuperação; não devolver estoque por timeout

Situação	Tratamento	Classificação
Cancelamento antes do início confirmado, após baixa	Bloquear início, confirmar não utilização, reverter baixa e cancelar orçamento	Compensação distribuída, detalhada abaixo
Peça já utilizada ou reparo iniciado	Bloquear reversão automática; registrar intervenção/devolução real quando aplicável	Limite físico; não inventar reposição
Pagamento recusado/expirado após reparo	Manter OS concluída, não quitada e sem entrega; permitir nova tentativa válida	Continuação financeira; reparo não é desfeito
Pagamento criado no provedor, mas resposta se perdeu	Billing consulta/concilia a tentativa; não cria outra cobrança às cegas	Recuperação de resultado desconhecido
Cobrança duplicada/indevida confirmada	Billing identifica a cobrança excedente, cancela ou estorna conforme resultado do provedor	Compensação financeira auditável
Evento SQS atrasou ou OS falhou ao registrar entrega	Outbox/reprocessamento ou repetir entrega após consultar quitação	Recuperação; não estornar pagamento correto por falha técnica

Retentativas esgotadas ficam pendentes com alerta, sem serem marcadas artificialmente como sucesso ou compensadas. E-mail enviado e serviço realizado não são apagados do passado: preservar a trilha.

5. Exemplo completo de compensação nos três serviços

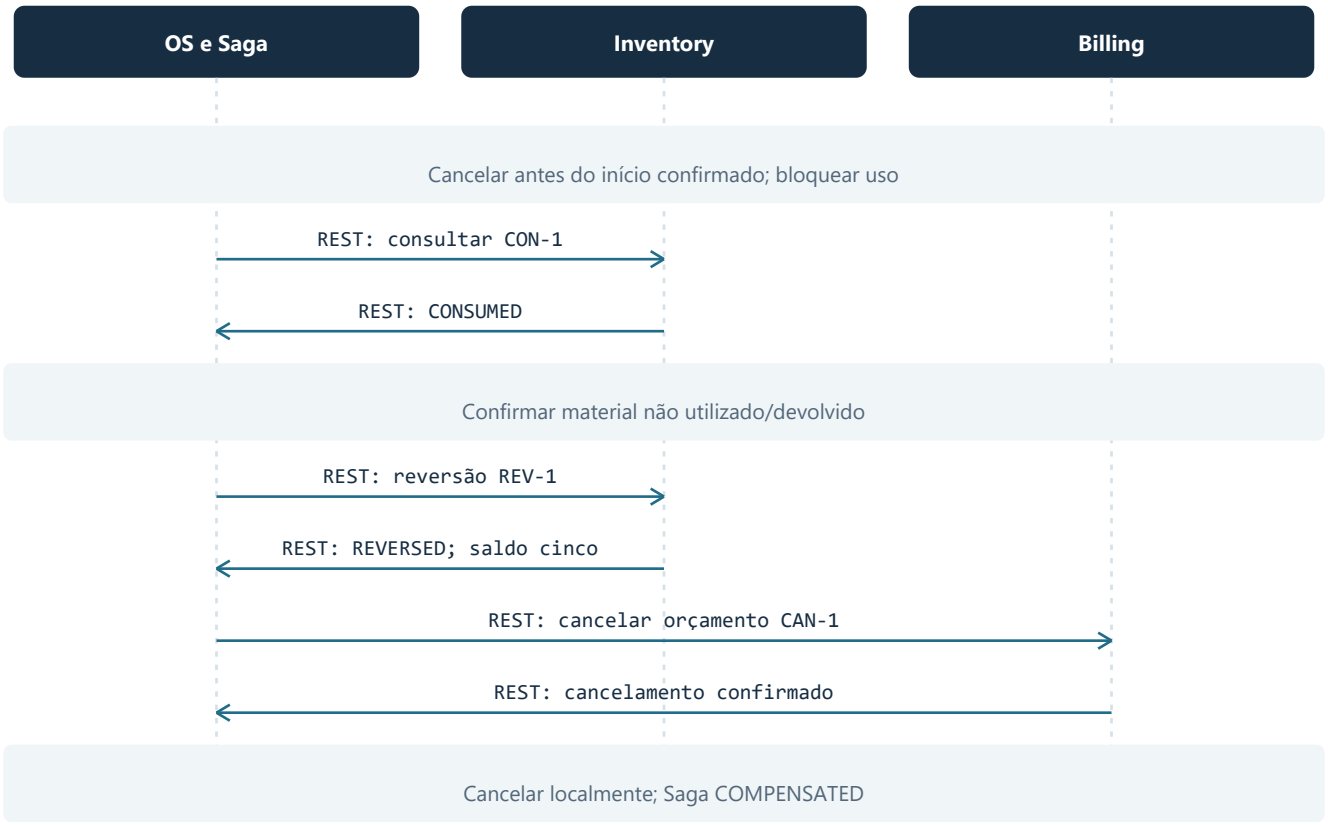
Cenário: cancelar antes de começar o reparo

O orçamento de R$ 350 já foi aprovado. A OS pediu CON-1; Inventory baixou um kit e ficou com quatro. A confirmação de início ainda não foi gravada na OS. A equipe solicita cancelamento antes de usar/retirar o material para o reparo.

Regra adicional a validar: a equipe só começa o trabalho e usa as peças após o início do item estar confirmado na OS. Se o material já saiu fisicamente, a devolução precisa ser confirmada; a aplicação não presume essa condição apenas por um timeout.

Passo	Responsável	Transação local / resultado
C1	OS / Matheus	Lock/versão bloqueia novos inícios e grava cancelamento + compensação pendente. Resposta tardia de CON-1 não pode iniciar o item
C2	OS → Inventory	Consulta resultado de CON-1; se ainda desconhecido, mantém pendência. Com não utilização/devolução validada, pede reversão com compensação REV-1
C3	Inventory / Anderson	Em uma transação, fecha a tentativa, grava REV-1, registra movimento inverso e muda CON-1 para REVERSED. Saldo quatro → cinco, uma única vez
C4	OS → Billing	Após reversão/ausência de efeito confirmada, pede cancelamento do orçamento vigente com operação CAN-1
C5	Billing / Julio	Cancela/invalida comercialmente o orçamento antes de uso; registra CAN-1 e resultado. Neste cenário não existe pagamento, pois o reparo não concluiu
C6	OS / Matheus	Com todos os resultados confirmados, cancela item/ordem, registra motivo e marca Saga COMPENSATED

Inventory não apaga a movimentação original. Billing não transforma cancelamento em rejeição do cliente nem apaga a aprovação anterior. O estado de cancelamento comercial é uma adição ao contrato a validar com Julio.



Se Inventory responder que o material foi utilizado, se algum item já teve início confirmado ou se Billing indicar efeito financeiro inesperado, o caso sai desse cancelamento automático e fica em intervenção/reconciliação. Não concluir C6 só porque o pedido de reversão foi enviado.

Se o processo cair após C3, OS retoma C4 pelo registro persistido. Repetir REV-1 não soma mais uma peça. Se a reversão chegar antes de CON-1, Inventory grava a tentativa encerrada para impedir que um consumo atrasado produza a baixa depois.

Exemplo financeiro: cobrança excedente

Depois do reparo, um pagamento correto já quitou R$ 350, mas outra cobrança de R$ 350 também foi confirmada. Billing identifica qual pagamento quita a OS e qual é excedente, persiste a operação de estorno e chama o provedor fora da transação SQL. Enquanto não souber o resultado, mantém estorno pendente e reconcilia.

Somente após confirmação final registra REFUNDED na cobrança excedente. O pagamento correto permanece PAID e a OS pode continuar quitada. Se a cobrança ainda não foi efetivada, o cancelamento depende da possibilidade e confirmação do provedor. Não assumir que um pedido de estorno foi concluído.

O atendimento acompanha a pendência, mas Billing executa e registra essa compensação. A autorização para solicitar estorno deve seguir a política financeira validada com Julio. Este exemplo não cria uma nova operação automática obrigatória para todo pagamento.

6. Estados globais e locais

Estado proposto da Saga na OS	Condição / próxima ação
IN_DIAGNOSIS	Aguardar composição e criação de orçamento pelo funcionário
WAITING_APPROVAL	Orçamento enviado; aguardar decisão
WAITING_REVISION	Orçamento rejeitado; aguardar revisão ou encerramento explícito
READY_FOR_EXECUTION	Orçamento vigente aprovado; ainda não significa pago
START_PENDING	Operação de consumo enviada, resultado ainda não confirmado localmente
IN_EXECUTION	Itens podem estar executando, concluídos ou aguardando peças; acompanhar operações por item
WAITING_PAYMENT	Reparo concluído; funcionário inicia pagamento sob demanda
READY_FOR_DELIVERY	Pagamento informado/confirmado; ainda consultar quitação ao entregar
COMPLETED	Entrega registrada e atendimento encerrado
COMPENSATING / COMPENSATED	Cancelamento em limpeza / todos os efeitos aplicáveis resolvidos
MANUAL_INTERVENTION	Efeito físico, financeiro ou resultado incerto exige tratamento explícito

Esses são estados técnicos propostos, distintos do enum comercial atual. Para vários itens, guardar o estado de cada operação: uma etapa global não substitui essa lista. WAITING_FOR_PARTS_AND_SUPPLIES e início pendente impedem conclusão indevida.

Billing guarda orçamento, pagamento e operações de cancelamento/estorno. Inventory guarda consumo, reversão e tentativa encerrada. A OS acompanha essas referências; não mantém uma cópia autoritativa dos bancos participantes.

7. O que precisamos acrescentar aos contratos

Eventos SQS: Billing publica, OS consome

Evento	Situação na proposta	Finalidade
OrcamentoAprovado / OrcamentoRejeitado	Já previstos por Julio	Atualizar decisão da revisão vigente
PagamentoConfirmado	Já previsto por Julio	Atualizar projeção de quitação
OrcamentoEnviado	Adição proposta	Resolver WAITING_APPROVAL com publicação durável
PagamentoAtualizado	Complemento proposto para reversões; outros estados a validar	Informar cancelamento/estorno e versão financeira; não deixar projeção eternamente PAID após reversão relevante

Usar eventId, eventType, schemaVersion, occurredAt, correlationId e IDs/versões do agregado, como no PDF. Acrescentar referência sagaId quando necessária à correlação, sem duplicar a máquina de estados nos participantes. Não ordenar decisões só pelo horário. Uma atualização financeira é por pagamento; settlement considera o conjunto e o valor devido, não apenas o último evento recebido.

REST: operações e recuperação

Contrato	Situação / dono
GET OS e quotation-context; POST consumo; POST pagamento; GET settlement	Preservar os contratos propostos pelo Julio
GET /internal/inventory/operations/{operationId}	Adição: Anderson fornece resultado persistido, incluindo UNKNOWN/REVERSED/CLOSED
POST /internal/inventory/consumptions/{operationId}/reversal	Adição: Anderson recebe compensationId e evidência de não utilização/devolução; fecha a tentativa
POST /internal/budgets/{budgetId}/cancellation	Adição: Julio recebe operationId, versão/motivo e devolve cancelamento confirmado ou impedimento
Consulta de operação financeira e cancelamento/estorno	Julio define contrato interno e elegibilidade; Billing reconcilia com Mercado Pago

Essas URIs novas são propostas para fechar com os donos. Não exigir mensagens SQS adicionais de Inventory para uma operação que o grupo decidiu fazer por REST. Se o primeiro request puder terminar rapidamente, responder normalmente; resultado ainda pendente precisa de consulta/operação durável, sem exigir que HTTP espere todo o atendimento.

8. Persistência, idempotência e recuperação

- OS grava intenção/operação e estado local antes do REST; depois grava o resultado com validação de versão/cancelamento. Worker retoma pendências após reinício. Nunca segurar lock/transação SQL durante a chamada remota.
- Inventory grava baixa, movimentos e resultado idempotente na mesma transação. Reversão grava movimento inverso, fechamento e resultado também juntos, usando lock/condição concorrente.
- Billing grava pagamento/operação externa antes da chamada ao provedor. Consulta/reconciliação resolve resultado desconhecido sem abrir nova cobrança automaticamente.
- Mesma operação e conteúdo repetidos não reaplicam efeito. Mesma chave com conteúdo diferente é conflito. Proteger também a mesma execução do item/tentativa contra outro UUID de duplo clique.
- Após REVERSED/CLOSED, consumo atrasado não pode reativar a operação. Resultado CONSUMED atrasado na OS também não pode vencer cancelamento já persistido. Nova tentativa só é criada explicitamente, com estado anterior resolvido e tentativa identificada.
- Para falta de saldo confirmada, registrar recusa sem baixa parcial. Após reposição pode haver nova tentativa explícita; timeout desconhecido não autoriza trocar a chave para tentar outra baixa.
- Billing grava alteração e evento na outbox local. OS grava inbox e transição local juntos; confirma SQS após commit. Reentrega/versionamento não dependem só de FIFO. Marcelo fecha fila Standard/FIFO, DLQ, visibilidade e permissões reais do Academy.
- GET settlement é uma confirmação no momento da consulta, não um commit conjunto com DELIVERED. Definir política para reversões financeiras concorrentes/excepcionais; não prometer atomicidade entre OS e Billing.

Outbox protege envio de eventos; registro de operação protege efeitos REST. Os dois podem usar workers da própria aplicação, sem framework novo ou quarto microsserviço.

9. Divisão para desenvolver e demonstrar

Membro	Entrega para este fluxo
Matheus	Estados globais, operações por item, início pendente, bloqueio de cancelamento/entrega, retomada e BDD
Julio	Eventos de orçamento/pagamento, cancelamento de orçamento, consultas financeiras, idempotência do provedor e estorno quando aplicável
Anderson	Consumo agrupado, consulta de operação, reversão auditável, fechamento contra comandos atrasados e disputa da última peça
Marcelo	URLs/autenticação interna, SQS/DLQ, bancos/permissões, deploy, traces e alarmes de operações pendentes

Roteiro sugerido para a apresentação:

1. Mostrar o fluxo feliz de R$ 350, destacando consumo antes do reparo e pagamento depois dele.
2. Mostrar a perda da resposta de CON-1: Inventory baixou uma vez; OS consulta e recupera sem segunda baixa.
3. Em outro atendimento, cancelar durante início pendente, antes de uso físico; mostrar reversão no Inventory, orçamento cancelado no Billing e Saga compensada na OS.
4. Mostrar pagamento recusado após reparo: ordem concluída, peças continuam consumidas e entrega bloqueada.
5. Mostrar entrega após settlement=true, mesmo com evento de pagamento atrasado; evidenciar logs/traces e estado nos três bancos.

10. Acordos para fechar na reunião

- □ Validar início pendente e regra de uso/retirada física; sem isso a reversão automática de estoque não é segura.
- □ Validar cancelamento de orçamento e revisão vigente; fechar P08/P13 e o snapshot dos preços.
- □ Fechar APIs/estados de consulta, reversão e impossibilidade; UNKNOWN nunca é sucesso ou recusa presumida.
- □ Fechar identidade dos IDs, erros, retenção, replay e fixtures comuns.
- □ Confirmar guarda de entrega, estornos e casos de OS cancelada/todos os itens cancelados, sem presumir valor zero ou cobrança integral.
- □ Manter NoSQL, Saga com compensação, BDD, 80%, Sonar, deploy e evidências no escopo da fase 4.

Referências

Base funcional: PDF **NumberOne_Fase4_Arquitetura_Referencia_Equipe**, Julio, v1.0, 01/11/2026, especialmente pp. 10-13, 22-28 e PF-06 na p. 30. Este texto propõe o complemento de Saga que aquele documento deixa para evolução; não atribui esses endpoints novos ao Julio.

- Revisão comparativa e checklist formal.
- Saga - AWS: coordenação e compensações entre transações locais.
- Outbox - AWS: persistência e intenção de publicação no mesmo commit.
- Idempotência - AWS: identidade persistente da operação e retentativas seguras.